

SIMATS ENGINEERING

Co3-ASSESSMENT TOOL 2

Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA0713

COURSE OUTCOME COVERED

CO3: Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions

Assessment Tool Weightage: 40%

Dr.V.Parthipan

QUESTIONS:4 Total Marks:40

Code of Conduct:

I levin anish.A (192511223) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A.Levin Anish

QUESTIONS:4 Total Marks:40

1. A multinational IT company uses a cloud-based video conferencing platform

for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an 8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

1. Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or the application layer.
2. Justify your answer based on the communication requirements of real-time multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

Case Study

A multinational IT company relies on a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reveals **8% packet loss, 180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the problem is related to either the transport protocol or the application-level buffering strategy.

1. Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or the application layer.

The issue is primarily caused by **network conditions and the application's handling of real-time media**, rather than the transport protocol alone. Most modern video conferencing applications such as Zoom, Microsoft Teams, and Google Meet use **UDP** for transmitting audio and video because UDP provides low latency and avoids delays caused by retransmission.

However, the observed **8% packet loss** and **180 ms jitter** significantly degrade communication quality. Packet loss causes missing audio samples and video frames, while high jitter results in packets arriving at irregular intervals. If the application's jitter buffer is not properly configured or adaptive enough, users experience voice distortion, frozen video, and delayed playback.

Therefore, although UDP does not guarantee packet delivery, it is the preferred transport protocol for real-time communication. The major issue lies in poor network performance combined with inadequate application-level buffering and error recovery mechanisms.

2. Justify your answer based on the communication requirements of real-time multimedia applications.

Real-time multimedia applications have strict requirements:

- **Low latency:** Audio and video must reach the receiver quickly to maintain natural conversation.
- **Minimal jitter:** Packets should arrive at regular intervals for smooth playback.
- **Tolerance to minor packet loss:** Small amounts of lost packets are preferable to delayed packets.
- **Continuous data flow:** Real-time communication prioritizes speed over perfect reliability.

These applications generally use **UDP** because:

- UDP does not require connection establishment.
- There are no retransmissions, reducing delay.
- Lower protocol overhead enables faster transmission.

Using **TCP** would increase latency because lost packets are retransmitted before subsequent packets are delivered. This would cause noticeable delays, making conversations unnatural. Therefore, UDP remains the most suitable transport protocol despite occasional packet loss.

3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

To improve conferencing quality, the following measures are recommended:

Protocol-Level Improvements

- Continue using **UDP** instead of TCP for audio and video transmission.
- Implement **Forward Error Correction (FEC)** to recover some lost packets without retransmission.
- Enable **Quality of Service (QoS)** to prioritize voice and video traffic over less time-sensitive traffic.
- Use **Adaptive Bitrate (ABR)** streaming to automatically adjust video quality based on available bandwidth.
- Optimize network routing to reduce congestion and packet loss.

Application-Level Improvements

- Configure an **adaptive jitter buffer** that adjusts buffer size based on network conditions.
- Use efficient video codecs such as **H.264**, **H.265**, or **VP9**, and audio codecs such as **Opus** to reduce bandwidth usage while maintaining quality.
- Enable packet loss concealment techniques to mask the effects of missing packets.
- Reduce video resolution or frame rate during periods of network congestion.

Conclusion

The conferencing issues are mainly caused by poor network performance, including **8% packet loss**, **180 ms jitter**, and fluctuating bandwidth, along with insufficient application-level buffering. The transport protocol (**UDP**) is appropriate for real-time multimedia because it minimizes latency. Replacing UDP with TCP would increase delays and reduce communication quality. Implementing QoS, adaptive bitrate streaming, Forward Error Correction, efficient codecs, and adaptive jitter buffering can significantly improve the overall performance and user experience of the cloud-based video conferencing system.

2. A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the average DNS lookup time is approximately 800 ms, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as Anycast DNS, DNS pre-fetching, and TTL optimization to reduce the lookup time below 50 ms.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

Case Study

A multinational software company hosts its web services across multiple continents. Employees frequently experience long delays before the application homepage loads. Network analysis shows that the average DNS lookup time is approximately **800 ms**, significantly affecting user experience. To improve performance and maintain continuous service availability during server failures, the organization plans to redesign its DNS infrastructure.

1. Analyze the reasons behind the high DNS resolution time.

The DNS resolution time of **800 ms** is much higher than the acceptable range (typically **20–50 ms**). Several factors may contribute to this delay:

- **Large geographical distance:** Users are accessing DNS servers located far from their region, increasing network latency.
- **High network congestion:** Heavy internet traffic during peak hours causes delayed DNS requests and responses.
- **Insufficient DNS servers:** A limited number of DNS servers leads to increased processing time and server overload.

- **Poor DNS caching:** Frequently requested DNS records are not cached effectively, forcing repeated recursive lookups.
- **High Time-to-Live (TTL) misconfiguration:** Very low TTL values increase the number of DNS queries, while excessively high TTL values delay updates after changes.
- **Slow recursive DNS resolution:** Multiple recursive lookups across root, TLD, and authoritative DNS servers increase lookup time.
- **Server failures or overload:** DNS servers experiencing failures or excessive load may respond slowly or become temporarily unavailable.

These factors collectively increase DNS resolution time and negatively impact webpage loading performance.

2. Propose a suitable DNS architecture using techniques such as Anycast DNS, DNS pre-fetching, and TTL optimization to reduce the lookup time below 50 ms.

Proposed DNS Architecture

To reduce DNS lookup time below **50 ms**, the company should implement the following architecture:

a) Anycast DNS

- Deploy multiple authoritative DNS servers across different continents.
- Configure all DNS servers to share the same Anycast IP address.
- Internet routing automatically directs users to the nearest available DNS server.
- If one server fails, traffic is automatically redirected to another healthy server without service interruption.

b) DNS Pre-fetching

- Enable DNS pre-fetching in web applications and browsers.
- Frequently accessed domain names are resolved before users click on links.
- Cached DNS information significantly reduces lookup delay for repeated requests.

c) TTL Optimization

- Set DNS TTL values to an optimal range (approximately **300–600 seconds**).
- Moderate TTL values allow DNS records to remain cached while still enabling timely updates.
- Frequently accessed records stay in local resolver caches, reducing repeated DNS queries.

d) Additional Improvements

- Deploy regional DNS servers close to users.
- Use high-performance recursive DNS resolvers.
- Implement DNS load balancing to distribute requests evenly across servers.

- Enable DNSSEC while ensuring optimized cryptographic performance.
- Continuously monitor DNS server health and performance.

With these techniques, the expected DNS lookup time can be reduced from **800 ms** to **less than 50 ms**.

3. Evaluate the advantages and limitations of the proposed architecture in terms of performance, scalability, and availability.

Advantages

Performance

- Users connect to the nearest DNS server, significantly reducing latency.
- DNS caching and pre-fetching minimize repeated DNS lookups.
- Faster DNS resolution improves webpage loading speed and user experience.
- Lower network congestion due to reduced recursive DNS queries.

Scalability

- New DNS servers can be added in different geographic regions as the organization grows.
- Anycast automatically distributes user requests across multiple servers.
- Load balancing prevents individual DNS servers from becoming overloaded.

Availability

- High availability through geographically distributed DNS servers.
- Automatic failover ensures uninterrupted service if one DNS server fails.
- Improved fault tolerance minimizes downtime during maintenance or network failures.

Limitations

- Initial deployment and infrastructure costs are relatively high.
- Anycast configuration requires advanced networking expertise.
- DNS pre-fetching may occasionally resolve domains that users never access, resulting in minor unnecessary network traffic.
- Incorrect TTL settings can either increase DNS query frequency (if too low) or delay propagation of DNS updates (if too high).
- Monitoring and managing multiple distributed DNS servers increases operational complexity.

Conclusion

The primary cause of the high **800 ms DNS resolution time** is inefficient DNS infrastructure, including distant DNS servers, poor caching, network congestion, and server overload. Implementing **Anycast DNS, DNS pre-fetching, TTL optimization**, regional DNS servers, and load balancing can reduce DNS lookup time to **below 50 ms**. This architecture provides excellent **performance, scalability, and high availability**, ensuring faster application access and uninterrupted service even during server failures. Although deployment costs and management complexity increase, the overall improvement in reliability and user experience makes this solution highly suitable for a multinational organization.

3. A cloud service provider delivers applications to millions of users worldwide.

During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly 800 ms because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

1. Examine the impact of centralized DNS architecture on application performance.
2. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
3. Analyze how TTL values, Anycast routing, and DNS caching influence system performance and fault tolerance.

Case Study

A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays while accessing cloud-hosted services. Investigation shows that DNS queries take nearly **800 ms** because all requests are directed to a **single primary DNS server**. The company requires a highly available distributed DNS solution capable of efficiently handling global traffic.

1. Examine the impact of centralized DNS architecture on application performance.

A centralized DNS architecture uses a single DNS server to process all DNS requests. Although it is simple to manage, it creates several performance and reliability issues, especially for global cloud services.

Impact on Application Performance

- **High DNS Lookup Time:** Users located far from the DNS server experience higher network latency, resulting in lookup times of around **800 ms**.

- **Server Overload:** During promotional events, millions of DNS requests are sent to the same server, causing heavy traffic and slower response times.
- **Single Point of Failure:** If the primary DNS server fails or becomes unreachable, users cannot resolve domain names, making cloud services inaccessible.
- **Network Congestion:** All DNS traffic travels through a single location, increasing bandwidth usage and congestion.
- **Poor User Experience:** Slow DNS resolution delays webpage loading, application startup, and API requests, reducing customer satisfaction.
- **Limited Scalability:** As the number of users increases, the single DNS server cannot efficiently handle the growing request volume.

Therefore, a centralized DNS architecture significantly reduces application performance, scalability, and reliability.

2. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.

Proposed Distributed DNS Architecture

To improve performance and availability, the company should deploy a globally distributed DNS architecture.

Components of the Solution

a) Anycast DNS

- Deploy multiple authoritative DNS servers across different continents.
- Assign the same Anycast IP address to all DNS servers.
- Internet routing automatically directs users to the nearest available DNS server.
- If one server fails, traffic is automatically rerouted to another healthy server.

b) Regional DNS Servers

- Install DNS servers in North America, Europe, Asia, Australia, and other major regions.
- Users receive responses from nearby servers, significantly reducing latency.

c) DNS Caching

- Enable recursive DNS caching at ISP and local DNS resolvers.
- Frequently requested domain names are stored locally, reducing repeated queries to authoritative servers.

d) TTL Optimization

- Configure DNS records with a moderate TTL value (300–600 seconds).
- Frequently accessed records remain cached while allowing timely updates when changes occur.

e) Load Balancing and Monitoring

- Distribute DNS requests evenly among multiple servers.
- Continuously monitor server health and automatically remove failed servers from service.
- Enable automatic failover for uninterrupted DNS resolution.

This architecture can reduce DNS lookup time from **800 ms** to **below 50 ms**, even during periods of heavy traffic.

3. Analyze how TTL values, Anycast routing, and DNS caching influence system performance and fault tolerance.

a) TTL (Time-To-Live)

Influence on Performance

- Higher TTL values allow DNS records to remain cached longer, reducing DNS queries and improving lookup speed.
- Lower TTL values increase the number of DNS lookups, creating additional network traffic.

Influence on Fault Tolerance

- Low TTL enables faster propagation of DNS changes during server failures.
- Very high TTL delays the update of DNS records, causing users to continue accessing failed servers.

Recommended TTL: 300–600 seconds for a balance between performance and flexibility.

b) Anycast Routing

Influence on Performance

- Routes users to the nearest available DNS server.
- Reduces network latency and improves response time.
- Distributes traffic across multiple DNS servers, preventing overload.

Influence on Fault Tolerance

- Provides automatic failover if a DNS server becomes unavailable.
- Eliminates the single point of failure.
- Improves service continuity during maintenance or network outages.

c) DNS Caching

Influence on Performance

- Stores previously resolved domain names locally.
- Reduces repeated DNS queries.
- Decreases DNS lookup time and network traffic.

Influence on Fault Tolerance

- Cached DNS records allow temporary operation even if authoritative DNS servers become unavailable.
- Excessive caching may delay updates if DNS records change before cache expiration.

Conclusion

The centralized DNS architecture causes high latency, server overload, poor scalability, and creates a single point of failure, resulting in DNS lookup times of approximately **800 ms**. A distributed DNS architecture using **Anycast DNS, regional DNS servers, DNS caching, TTL optimization, and load balancing** can reduce lookup latency to **below 50 ms** while providing high availability and fault tolerance. This solution ensures faster application access, improved scalability, and uninterrupted service for millions of users worldwide, even during peak traffic periods.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the average order acknowledgment latency increases from 1 ms to 40 ms, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

1. Analyze three possible TCP-related factors responsible for the latency increase, considering flow control, Nagle's algorithm, and SYN queue limitations.
2. Explain how each factor affects transaction processing in high-frequency systems
3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

Case Study

A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are processed every second. During market opening hours, the average order acknowledgment latency increases from **1 ms** to **40 ms**, causing delayed trade confirmations and customer dissatisfaction. Network monitoring reveals increased **TCP retransmissions**, longer **connection establishment times**, and excessive **buffering**.

1. Analyze three possible TCP-related factors responsible for the latency increase, considering flow control, Nagle's algorithm, and SYN queue limitations.

a) TCP Flow Control

TCP uses a **sliding window mechanism** to prevent the receiver from becoming overloaded. During peak trading hours, the receiver's buffer may become full due to the high volume of incoming orders. As a result, the receiver advertises a smaller receive window, forcing the sender to slow down data transmission. This increases waiting time and acknowledgment latency.

b) Nagle's Algorithm

Nagle's algorithm improves network efficiency by combining multiple small packets into a larger packet before transmission. However, stock trading systems send many very small messages that require immediate delivery. Waiting for additional packets before transmission introduces unnecessary delay, increasing acknowledgment latency.

c) SYN Queue Limitations

Each incoming TCP connection request is temporarily stored in the server's SYN queue until the three-way handshake is completed. During market opening, thousands of clients attempt to connect simultaneously. If the SYN queue becomes full, new connection requests are delayed or dropped, resulting in longer connection establishment times and additional retransmissions.

2. Explain how each factor affects transaction processing in high-frequency systems.

TCP Flow Control

- Slows the transmission of buy and sell orders when receiver buffers become full.
- Increases acknowledgment delay from milliseconds to tens of milliseconds.
- May delay order execution, affecting trading accuracy and profitability.

Nagle's Algorithm

- Delays transmission of small trade messages while waiting to combine packets.
- Adds unnecessary latency to time-sensitive transactions.
- Reduces the responsiveness required in high-frequency trading systems.

SYN Queue Limitations

- Slows the establishment of new TCP connections during peak load.
- Causes repeated SYN retransmissions if the queue is full.
- Delays client access to the trading platform and increases transaction processing time.

Overall, these TCP-related issues reduce the speed and reliability required for high-frequency financial transactions.

3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

a) Disable Nagle's Algorithm

- Enable the **TCP_NODELAY** socket option.
- Sends small packets immediately without waiting for packet aggregation.
- Significantly reduces latency for trading messages.

b) Increase TCP Buffer Sizes

- Increase TCP send and receive buffer sizes to handle higher traffic volumes.
- Prevent receiver buffer overflow and reduce flow-control delays.

c) Increase SYN Queue Size

- Configure a larger SYN backlog to accommodate more simultaneous connection requests.
- Reduces connection establishment delays during peak trading periods.

d) Enable SYN Cookies

- Protects the server from SYN queue overflow.
- Maintains reliable connection establishment under heavy load.

e) Use Persistent TCP Connections

- Reuse existing TCP connections instead of creating a new connection for every trade.
- Eliminates repeated three-way handshakes and reduces connection setup time.

f) Optimize TCP Retransmission Parameters

- Configure appropriate retransmission timeout (RTO) values.
- Reduces unnecessary retransmissions while maintaining reliability.

g) Improve Network Infrastructure

- Upgrade bandwidth and deploy Quality of Service (QoS) to prioritize trading traffic.
- Reduce congestion and packet loss, improving overall transaction performance.

Conclusion

The increase in order acknowledgment latency from **1 ms to 40 ms** is primarily caused by **TCP flow control**, **Nagle's algorithm**, and **SYN queue limitations** under heavy market traffic. These factors slow transaction processing by delaying packet transmission, increasing connection setup time, and causing retransmissions. By disabling Nagle's algorithm, increasing TCP buffer sizes and SYN queue capacity, enabling SYN cookies, using persistent TCP connections, and optimizing network infrastructure, the financial institution can significantly reduce latency, improve transaction reliability, and provide faster trade confirmations during peak trading hours.